

Lightweight Fault Detection and Localisation in AWS Serverless Microservices

Yashaswini Penumarthy | Student ID: 24262404

MSc Cloud Computing — National College of Ireland — Short Explanation Guide

Baseline: Xing et al. (2025) Sensors — DOI 10.3390/s25113396

Read aloud in simple English. Numbers come from STATUS.md / results — do not invent live AWS metrics.

1. What is the project?

This project builds a small serverless online-shop backend on Amazon Web Services and studies how well simple CloudWatch rules plus X-Ray ranking can detect and localise faults — compared with heavy deep-learning detectors that need lots of labelled training data.

The artefact is a four-function order path: orders-api, inventory, payments, and notify. Traffic enters through API Gateway. State lives in DynamoDB. An SSM Parameter Store switch injects faults on purpose. A lightweight detector watches CloudWatch metrics and X-Ray traces.

Three research legs: (1) Xing et al. as published accuracy ceiling (not same-rig); (2) rules versus RCAEval baselines on the same ninety public RE2-OB cases; (3) live AWS lite tracing overhead — full versus policy versus off. The live stack was destroyed after the round.

Pitch: “I study how well simple CloudWatch rules plus X-Ray ranking can find faults in a serverless shop, compared with heavy deep models, and what continuous tracing costs.”

2. Problem statement + real-life example

When a serverless checkout slows down, the customer only sees the front door fail. The root cause may be an inventory timeout, a payment throttle, or a slow notify path. Deep detectors need large labelled telemetry corpora that a newly deployed, cost-constrained service does not have.

Practical question: how good is a no-training method that only uses CloudWatch alarms and X-Ray traces — and what does continuous monitoring cost in volume, latency, and money when accuracy and overhead are measured together?

Real-life example — pizza shop. Orders-api is the counter; inventory is the kitchen; payments is the card terminal; notify is the SMS. If the kitchen freezes, customers still complain at the counter. A fancy AI doctor needs thousands of past failures; a new small clinic cannot afford that. This project is the nurse with a checklist (CloudWatch rules) and a corridor map (X-Ray ranking).

Why serverless makes it hard: functions are short-lived; you pay for every log line and trace segment; faults can be timeout, error, throttle, or slow; symptoms often appear at the

entry service even when the root cause is downstream.

3. Baseline paper

Xing, Wang and Liu (2025), Sensors 25(11):3396 (DOI 10.3390/s25113396). Dual-channel deep model (TCN + VAE with contrastive learning and causal inference) for microservice anomaly detection and fault localisation.

Verified claims: detection $F1 \approx 0.938$, accuracy ≈ 0.954 , fault-component localisation precision ≈ 0.876 under ideal labelled-data conditions.

Gap: needs training data and compute; does not jointly report continuous monitoring overhead on AWS serverless. We use Xing as a published accuracy ceiling — not a same-rig re-run. Like-for-like work uses RCAEval (Pham et al., 2025).

4. How we solve it / research question

RQ: What accuracy–overhead position does rule-based fault detection and localisation occupy relative to learned deep baselines in AWS serverless microservices?

Solution: freeze CloudWatch rules + X-Ray dependency ranker (no training); compare on RCAEval RE2-OB; measure live tracing overhead (full / policy / off). Competitive only if $F1$ within 10 pp of strongest baseline AND telemetry volume cut by $\geq 50\%$.

Honest STATUS: rule $F1 = 0.469$ (not within 10 pp of Xing 0.938); $AC@3 = 0.611$; policy vs full volume reduction ≈ 0.803 . CausalRCA $n=4$ is quarantined.

What you must NOT claim: that you beat Xing on the same rig; that CausalRCA is the strongest baseline; that lite Leg 3 is a full 30-minute confirmatory design; that the live stack is still running.

5. Dataset / data

Two sources — synthetic or public. No real customer PII. Own AWS account; destroy-after.

- RCAEval RE2-OB (Leg 2): public microservice fault cases (Pham et al., 2025). Online Boutique-style telemetry. Ninety cases for rules, BARO, CIRCA, TraceRCA, hybrid — same data for every method.
- Live AWS lite (Leg 3): synthetic HTTP at 1 rps for 5 minutes × three tracing conditions (≈300 requests/condition) on faultlab in eu-west-1.
- Faults injected via SSM switches (timeout, error, throttle, latency). Ground truth = injection schedule, not guessed from logs.
- Not in scope: production traffic, multi-region estates, full 90-case parquet catalogue for learned lower bound (subset n=8 only).

6. Implementation + technologies / AWS services

- AWS Lambda — four functions (orders, inventory, payments, notify).
- API Gateway — REST entry for shop traffic.
- DynamoDB — order / inventory state.
- CloudWatch — metrics + alarms for the rule detector.
- AWS X-Ray — traces for localisation; sampling full / policy / off.
- SSM Parameter Store — fault injection switches.
- SAM + Terraform — deploy/destroy faultlab (destroyed after lite run).
- Python 3 + pytest/moto — 76 unit tests; RCAEval offline eval scripts.

7. End-to-end flow

- Deploy faultlab (API + 4 Lambdas + DynamoDB + X-Ray + SSM).
- Warm shop; start synthetic load.
- Flip SSM fault switches on a schedule; log ground-truth windows.
- CloudWatch rules fire → detection.
- X-Ray ranker scores root services → localisation.
- Offline: rules + RCAEval baselines on 90 RE2-OB cases → AC@k / F1.
- Live lite: full vs policy vs off → results/live/overhead.json.
- Destroy stack; keep results/rcaeval/ and results/live/ only.

8. Exact speaking script (dataset + implementation + technologies)

“My dataset has two parts. First, the public RCAEval RE2-OB fault cases — ninety labelled microservice failures I use offline so my rule detector and published baselines see the same data. Second, synthetic live traffic on my own AWS account: about one request per second for five minutes under three tracing settings. I never use real customer data.”

“I implemented a four-function serverless shop with Lambda, API Gateway, DynamoDB, CloudWatch, X-Ray, and an SSM fault switch. Infrastructure is SAM and Terraform. The detector is frozen CloudWatch rules plus an X-Ray ranker — no model training. After the lite overhead round I destroyed the stack. Technologies are Python, AWS managed services, and the RCAEval benchmark tooling.”

9. STATUS snapshot + five Q&A

CA2 / gate	COMPLETE; INITIAL_EVAL_PASS=yes; Final-3 done
Rule F1 / AC@3	0.469 / 0.611 (STATUS Leg 2)
Overhead	policy vs full volume reduction $\approx$ 0.803 (lite)
AWS residual	closed — LITE_DESTROYED=yes

Q: Is your data real customer traffic?

A: No — public RCAEval cases plus synthetic load on my own AWS account.

Q: Did you beat Xing's F1 of 0.938?

A: No. Rule F1 is 0.469; Xing is a published ceiling, not same-rig.

Q: What AWS services did you use?

A: Lambda, API Gateway, DynamoDB, CloudWatch, X-Ray, SSM; SAM/Terraform.

Q: Is the live stack still running?

A: No — lite Leg 3 measured then destroyed.

Q: Localisation versus detection?

A: Detection = something is wrong; localisation = which service is the root cause.